

Soal Latihan IBDA2011

Modul 1: Dasar Analisis dan Perancangan Algoritma

Dipersiapkan oleh : Yozef Tjandra

Soal 1.

[Masalah & Instance]

Diberikan dua pernyataan berikut:

1. “Urutkan array $[7, 2, 9, 1, 5]$ secara menaik” adalah sebuah masalah, sedangkan “Urutkan sembarang array berisi n bilangan real secara menaik” adalah sebuah instance.
 2. Suatu algoritma dapat dikatakan telah menyelesaikan masalah “Carilah elemen maksimum dari suatu array bilangan bulat” apabila algoritma tersebut terbukti memberikan jawaban benar untuk semua array yang panjangnya genap.
- (a) Tentukan nilai kebenaran (benar/salah) dari masing-masing pernyataan di atas.
- (b) Berikan penjelasan untuk setiap jawaban Anda pada poin (a).

Soal 2.

[Masalah & Instance]

Seorang mahasiswa merancang sebuah algoritma untuk masalah “Carilah bilangan prima terbesar yang kurang dari n , untuk n bilangan bulat positif”. Ia menguji algoritmanya pada tiga instance: $n = 10$, $n = 20$, dan $n = 50$, dan pada ketiganya algoritma memberikan jawaban yang benar. Berdasarkan hasil ini, mahasiswa tersebut menyimpulkan bahwa algoritmanya sudah benar (valid) untuk menyelesaikan masalah tersebut.

- (a) Identifikasi mana yang merupakan “masalah” dan mana yang merupakan “instance” dalam skenario di atas.
- (b) Apakah kesimpulan mahasiswa tersebut dapat dibenarkan? Jelaskan keterkaitannya dengan konsep validitas suatu algoritma.

Soal 3.

[Validitas Algoritma]

Diberikan masalah: “Carilah elemen terbesar dari suatu array A yang berisi n bilangan bulat ($n \geq 1$), dengan indeks array dimulai dari 0.”

```
1 FindMax(A, n):  
2   max ← -1  
3   for i ← 0 to n - 1 do  
4       if A[i] > max then  
5           max ← A[i]  
6   return max
```

- (a) Tunjukkan bahwa algoritma ini gagal memberikan output yang benar untuk beberapa instance.
- (b) Identifikasi seluruh instance yang menyebabkan algoritma ini gagal.

Soal 4.

[Validitas Algoritma]

Diberikan masalah: “Carilah indeks elemen x pada array A yang telah terurut menaik dan berisi n bilangan bulat ($n \geq 1$), dengan indeks array dimulai dari 0; jika x tidak ditemukan, kembalikan -1 .”

```

1 BinarySearch(A, n, x):
2   low ← 0
3   high ← n
4   while low ≤ high do
5       mid ← floor((low + high) / 2)
6       if A[mid] = x then
7           return mid
8       else if A[mid] < x then
9           low ← mid + 1
10      else
11          high ← mid - 1
12  return -1

```

- Tunjukkan bahwa algoritma ini gagal untuk beberapa instance.
- Identifikasi seluruh instance yang menyebabkan algoritma ini gagal.

Soal 5.

[Validitas Algoritma]

Diberikan masalah: “Hitung $a \bmod b$ (sisanya pembagian a oleh b) untuk bilangan bulat a bebas dan b bilangan bulat positif, menggunakan pengurangan berulang.”

```

1 ModManual(a, b):
2   while a ≤ b do
3       a ← a - b
4   return a

```

- Tunjukkan bahwa algoritma ini tidak berhenti untuk beberapa instance.
- Identifikasi seluruh instance (a, b) yang menyebabkan algoritma ini tidak berhenti.

Soal 6.

[Validitas Algoritma]

Diberikan masalah: “Tentukan apakah suatu bilangan bulat positif n merupakan bilangan berbentuk 2^k , untuk suatu bilangan bulat $k \geq 0$.”

```

1 IsPowerOfTwo(n):
2   pow ← 1
3   while pow ≠ n do
4       pow ← pow * 2
5   return True

```

- Tunjukkan bahwa algoritma ini tidak berhenti untuk beberapa instance.
- Identifikasi seluruh instance yang menyebabkan algoritma ini tidak berhenti.

Soal 7.

[Validitas Algoritma]

Masalah: Suatu panitia study tour perlu mengangkut N peserta menggunakan minibus berkapasitas maksimum 18 orang (minibus boleh tidak terisi penuh). Berapa jumlah minibus minimum yang dibutuhkan?

```

1 MinBuses(N):
2   if N = 0 then
3       return 0
4   buses ← 0
5   remaining ← N
6   while remaining > 0 do

```

```

7     buses ← buses + 1
8     remaining ← remaining - 18
9     return buses

```

- (a) Buktikan bahwa algoritma di atas valid (memberikan jawaban minimum yang benar) untuk semua $N \geq 0$.
- (b) Rancang ulang algoritma di atas agar lebih efisien, sehingga hanya membutuhkan waktu $O(1)$.

Soal 8.

[Notasi Asimtotik]

Diketahui $f(n) = 4n^3 + 2n^2 - 7$. Manakah dari pernyataan berikut yang **salah**?

- A. $f(n) = O(n^4)$
- B. $f(n) = O(n^3)$
- C. $f(n) = \Theta(n^3)$
- D. $f(n) = \Omega(n^2)$
- E. $f(n) = \Omega(n^4)$

Jelaskan alasan Anda memilih jawaban tersebut.

Soal 9.

[Notasi Asimtotik]

Diketahui $f(n) = n \log n + n^2$. Manakah dari pernyataan berikut yang **salah**?

- A. $f(n) = O(n^3)$
- B. $f(n) = O(n^2)$
- C. $f(n) = \Theta(n^2)$
- D. $f(n) = \Omega(n \log n)$
- E. $f(n) = \Theta(n \log n)$

Jelaskan alasan Anda memilih jawaban tersebut.

Soal 10.

[Worst/Best Case]

Diberikan potongan kode berikut untuk mencari posisi kemunculan pertama elemen x pada array A berukuran n , dengan penelusuran **dimulai dari indeks terakhir menuju indeks pertama**:

```

1 ReverseLinearSearch(A, n, x):
2   for i ← n-1 downto 0 do
3     if A[i] = x then
4       return i
5   return -1

```

Anggap operasi dasar adalah perbandingan pada baris 2.

- (a) Untuk suatu ukuran n tertentu, tentukan karakteristik instance (array A dan nilai x) yang menghasilkan **worst case**, beserta jumlah operasi dasar yang dilakukan.
- (b) Tentukan karakteristik instance yang menghasilkan **best case**, beserta jumlah operasi dasar yang dilakukan.

Soal 11.*[Worst/Best Case]*

Diberikan potongan kode berikut untuk menghitung banyaknya digit dari suatu bilangan bulat positif m :

```
1 CountDigits(m):
2   count ← 0
3   while m > 0 do
4       count ← count + 1
5       m ← floor(m / 10)
6   return count
```

Anggap operasi dasar adalah operasi pembagian pada baris 4, dan anggap ukuran input n adalah banyaknya digit dari m (bukan nilai m itu sendiri).

Seorang mahasiswa menyatakan: “Instance dengan $m = 1$ adalah best case dari algoritma ini, karena jumlah operasi dasarnya paling sedikit dibandingkan nilai m lainnya.”

- (a) Jelaskan mengapa pernyataan mahasiswa tersebut **keliru**, dikaitkan dengan fakta bahwa best case harus dinilai secara asimtotik.
- (b) Jelaskan bagaimana karakteristik instance worst case dan best case dari algoritma ini untuk suatu ukuran n (banyaknya digit) tertentu yang tetap, dan berikan alasannya.

Soal 12.*[Analisis Nonrekursif]*

Diberikan potongan kode berikut, dengan input array A berukuran n :

```
1 CountInversions(A, n):
2   count ← 0
3   for i ← 0 to n-1 do
4       for j ← i+1 to n-1 do
5           if A[i] > A[j] then
6               count ← count + 1
7   return count
```

- (a) Tentukan operasi dasar dari algoritma di atas, lalu turunkan secara formal (menggunakan notasi sigma/sumasi) fungsi kompleksitas waktu $T(n)$, dan nyatakan hasilnya dalam notasi Θ .
- (b) Tentukan kompleksitas ruang (space complexity) dari algoritma tersebut, dengan menyebutkan variabel apa saja yang berkontribusi.

Soal 13.*[Analisis Nonrekursif]*

Diberikan potongan kode berikut, dengan input bilangan bulat positif n :

```
1 SumUpTo(n):
2   i ← 1
3   sum ← 0
4   while i <= n do
5       sum ← sum + i
6       i ← i + 1
7   return sum
```

- (a) Tentukan operasi dasar dari algoritma di atas, lalu turunkan secara formal fungsi kompleksitas waktu $T(n)$, dan nyatakan hasilnya dalam notasi Θ .
- (b) Tentukan kompleksitas ruang (total: input + auxiliary) dari algoritma tersebut.

Soal 14.*[Analisis Rekursif]*

Diberikan potongan kode rekursif berikut, dengan input bilangan bulat positif n :

```
1 CountDivisionsRecursive(n):  
2   if n = 1 then  
3       return 0  
4   else  
5       return 1 + CountDivisionsRecursive(floor(n / 3))
```

- (a) Tuliskan relasi rekurensi $T(n)$ untuk kompleksitas waktu algoritma ini (termasuk kondisi basis), lalu selesaikan secara formal (misalnya dengan metode backward substitution, dengan mengasumsikan $n = 3^k$ untuk mempermudah) untuk mendapatkan $T(n)$ dalam notasi Θ .
- (b) Tentukan kompleksitas ruang (total: input + auxiliary, termasuk call stack) dari algoritma tersebut.

Soal 15.*[Analisis Rekursif]*

Diberikan potongan kode rekursif berikut untuk mencari elemen terbesar pada array A berukuran n (dengan indeks 0 hingga $n - 1$):

```
1 FindMaxRecursive(A, n):  
2   if n = 1 then  
3       return A[0]  
4   else  
5       m ← FindMaxRecursive(A, n - 1)  
6       if A[n-1] > m then  
7           return A[n-1]  
8       else  
9           return m
```

- (a) Tuliskan relasi rekurensi $T(n)$ untuk kompleksitas waktu algoritma ini (termasuk kondisi basis), lalu selesaikan secara formal untuk mendapatkan $T(n)$ dalam notasi Θ .
- (b) Tentukan kompleksitas ruang (total: input + auxiliary, termasuk call stack) dari algoritma tersebut.

Soal 16.*[Master Theorem]*

Diberikan suatu algoritma divide-and-conquer dengan relasi rekurensi berikut:

$$T(n) = 4T(n/2) + n, \quad T(1) = 1$$

Gunakan Master Theorem untuk menentukan $T(n)$ dalam notasi Θ . Tunjukkan langkah-langkah penerapannya (identifikasi a , b , $f(n)$, hitung $n^{\log_b a}$, lalu bandingkan dengan $f(n)$ untuk menentukan kasus mana yang berlaku).

Soal 17.*[Master Theorem]*

Diberikan suatu algoritma divide-and-conquer dengan relasi rekurensi berikut:

$$T(n) = 2T(n/2) + n, \quad T(1) = 1$$

Gunakan Master Theorem untuk menentukan $T(n)$ dalam notasi Θ . Tunjukkan langkah-langkah penerapannya (identifikasi a , b , $f(n)$, hitung $n^{\log_b a}$, lalu bandingkan dengan $f(n)$ untuk menentukan kasus mana yang berlaku).